★ 실습 강의 · 멀티 AI 팀

Claude Code 멀티 AI 팀 실습

Claude가 만들고 GPT가 검토하는 나만의 AI 팀 구성

Codex CLI · Gemini CLI · MCP · /team · 자동차 적용 예제(Day 4·5)

수강생은 프롬프트를 붙여 넣고 결과를 확인만 한다.
설치, 등록, 오류 해결은 모두 Claude Code에게 시킨다.

오늘의 실습은 세 가지를 남긴다

AI 하나로 만들고 끝내던 작업에 검토자를 붙이는 과정이다

1

다른 AI를 Claude Code의 도구로 등록할 수 있다

MCP로 GPT(Codex)를 연결하면 Claude가 직접 검토를 요청한다.

2

분업 규칙은 슬래시 명령 하나로 저장한다

/team 한 줄이면 작성, 검토, 반영, 보고가 자동으로 이어진다.

3

검토자의 지적도 틀릴 수 있다

타당한 제안만 반영하고, 최종 판단은 사람이 한다.

실습 내내 수강생의 일은 붙여 넣기와 확인뿐이다

MCP는 AI에게 도구를 꽂는 표준 규격이다

MCP = Model Context Protocol, Anthropic이 2024년 공개한 개방형 규약

Claude Code가 부를 수 있는 '도구'를 외부 프로그램이 제공하는 방식

무엇인가

- AI용 USB 포트에 비유할 수 있다
- 도구 제공자를 MCP 서버라 부른다
- claude mcp add 명령으로 등록한다

왜 쓰는가

- Codex CLI가 MCP 서버를 내장하고 있다
- 등록하면 Claude가 GPT를 직접 부른다
- 복사·붙여넣기 왕복이 사라진다

작성자와 검토자를 나누면 오류를 더 잘 잡는다

사람의 코드 리뷰(code review)와 같은 원리 — 만든 사람은 자기 실수를 못 본다

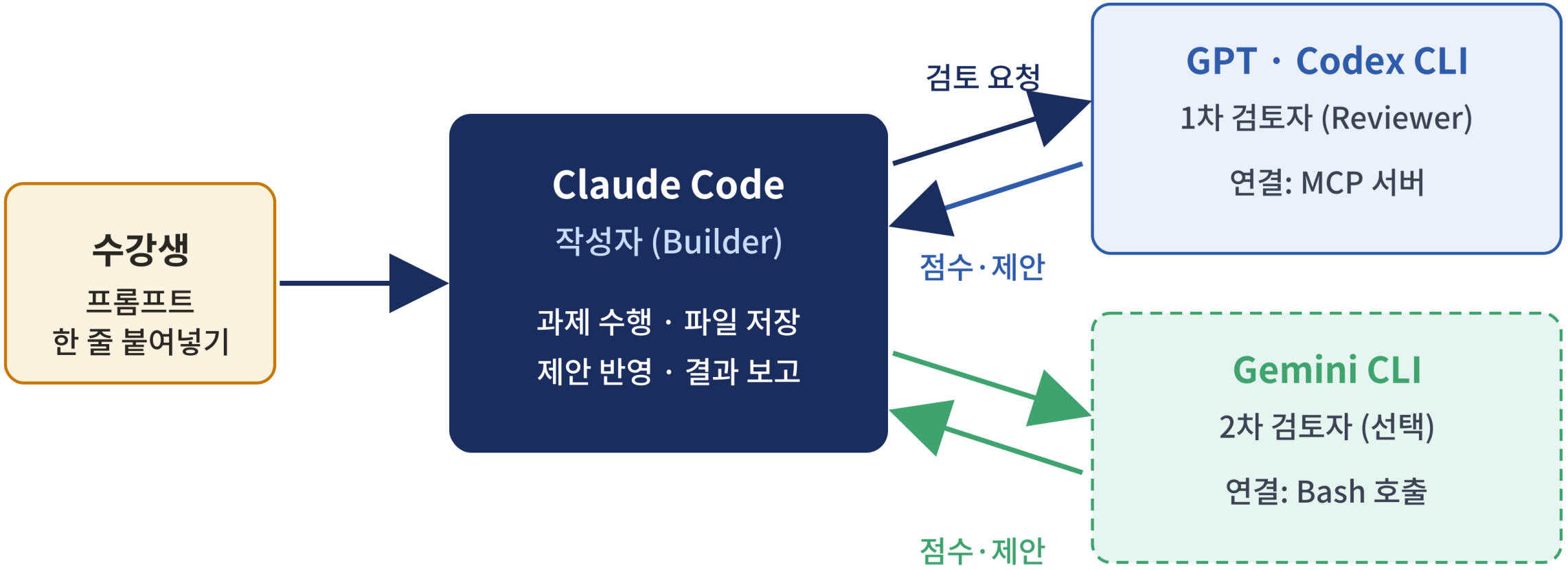
역할	AI	연결 방식
작성자 (Builder)	Claude Code	본체
1차 검토자 (Reviewer)	GPT (Codex CLI)	내장 MCP 서버 (codex mcp-server)
2차 검토자 (선택)	Gemini (Gemini CLI)	비대화형 실행 (gemini -p)을 Bash로 호출

권장 구성

- 처음에는 Claude와 GPT 두 명으로 시작한다. 검토자가 늘수록 의견 충돌과 비용만 커진다.

Claude Code가 지휘자, 다른 AI는 검토 도구다

수강생은 Claude에게만 말하고, Claude가 GPT와 Gemini에게 검토를 요청한다



준비물은 계정 두 개와 Claude Code뿐이다

Node.js가 없어도 된다 — 실습 1에서 Claude Code가 함께 설치한다

준비물	용도	비고
Claude Code	작성자, 설치 담당	이미 설치되어 있어야 한다
ChatGPT 계정 (Plus 이상)	Codex CLI 로그인	구독 사용량 안에서 검토
Google 계정	Gemini CLI 로그인	무료 사용량 안에서 사용
Node.js 18 이상	CLI 설치 도구 (npm)	없으면 Claude가 설치

API 키 방식도 가능하지만 호출마다 과금되므로 이 실습은 계정 로그인 방식을 쓴다

01

실습 1

GPT(Codex)를 검토자로 연결

설치부터 MCP 등록, 연결 확인까지 Claude Code에게 맡긴다

프롬프트 한 장으로 설치와 등록을 끝낸다

명령어가 바뀌어도 되도록 공식 문서를 먼저 확인하게 시킨다

[입력] 프롬프트 · Claude Code

```
OpenAI Codex CLI를 내 PC에 설치하고 Claude Code의 MCP 도구로 연결해줘.

1. 공식 문서를 검색해서 지금 기준의 설치 명령과 MCP 서버 실행 명령(codex mcp-server)을 확인해줘.
2. Node.js가 없으면 먼저 설치해줘.
3. npm으로 Codex CLI를 전역 설치해줘.
4. codex login을 실행해서 내가 브라우저에서 ChatGPT 계정으로 로그인하게 안내해줘.
   로그인이 끝나면 내가 "완료"라고 말할게.
5. claude mcp add 명령으로 codex를 user 범위로 등록해줘.
6. claude mcp list로 연결 상태(Connected)를 확인하고 결과를 보여줘.
오류가 나면 원인을 찾아 직접 고치고, 무엇을 고쳤는지 한 줄로 알려줘.
```

사람이 움직이는 곳은 4단계 하나뿐이다

- 브라우저 로그인은 본인이 직접 하고, "완료"라고 입력한다

Connected가 보이면 연결은 끝났다

확인 후 Claude Code를 한 번 재시작해야 새 도구가 목록에 나타난다

[출력] 결과 · claude mcp list

```
Checking MCP server health...

codex: codex mcp-server - ✓ Connected
```

[입력] 프롬프트 · Claude Code (재시작 후)

```
codex 도구로 GPT에게 "피보나치 수열을
구하는 가장 빠른 방법"을 물어보고, 답을
그대로 보여준 뒤 네 의견을 덧붙여줘.
```

무엇을 확인하나

- GPT의 답과 Claude의 의견이 따로 표시되는지 본다 — 두 AI가 실제로 대화한 증거다
- Failed가 보이면 다음 장의 문제 해결 프롬프트를 그대로 붙여 넣는다

연결 오류도 Claude Code가 고친다

수강생이 터미널 오류를 해석할 필요는 없다

[입력] 프롬프트 · Claude Code

```
claude mcp list에서 codex가 연결되지 않아. 원인을 찾아서 고치고, 고친 뒤 다시 확인해줘.
```

자주 나오는 증상	흔한 원인
codex: command not found	전역 설치 경로가 PATH에 없다
Failed to connect	codex login을 하지 않았거나 만료됐다
도구 목록에 codex가 없다	Claude Code를 재시작하지 않았다

02

실습 2 · 선택

Gemini를 두 번째 검토자로

별도 MCP 서버 없이 gemini -p 한 줄을 Bash로 부른다

Gemini는 Bash 호출만으로 검토자가 된다

gemini -p "질문" = 대화창 없이 답 하나만 받는 비대화형 실행

[입력] 프롬프트 · Claude Code

```
Google Gemini CLI를 설치하고 Claude Code에서 쓸 수 있게 해줘.

1. 공식 저장소(google-gemini/gemini-cli)에서 지금 기준 설치 명령과 비대화형 실행 옵션을 확인해줘.
2. npm으로 전역 설치해줘.
3. 처음 로그인이 필요하면 gemini를 한 번 실행해서 내가 Google 계정으로 로그인하게 안내해줘.
   끝나면 "완료"라고 말할게.
4. gemini -p "안녕" 처럼 비대화형으로 답이 오는지 확인해줘.
5. 앞으로 "Gemini에게 물어봐"라고 하면 이 방식으로 호출하라는 내용을 CLAUDE.md에 추가해줘.
```

확인: 4단계에서 Gemini의 인사말이 출력되면 성공이다

CLAUDE.md는 Claude Code가 매번 먼저 읽는 메모장이다

프로젝트 폴더에 두면 대화를 새로 시작해도 규칙이 유지된다

[출력] Claude가 추가 · CLAUDE.md

```
## 외부 AI 호출 규칙
- "Gemini에게 물어봐" → Bash로 gemini -p "..." 실행
- "GPT에게 검토" → codex MCP 도구 사용
- 기밀·개인정보는 외부 AI에 보내지 않는다
```

내용은 예시다 — 실제 문장은 Claude가 작성한다

왜 필요한가

- 매번 호출 방법을 설명하지 않아도 된다
- 보안 규칙도 함께 적어 둔다
- 직접 쓰지 않고 Claude에게 쓰게 한다

03

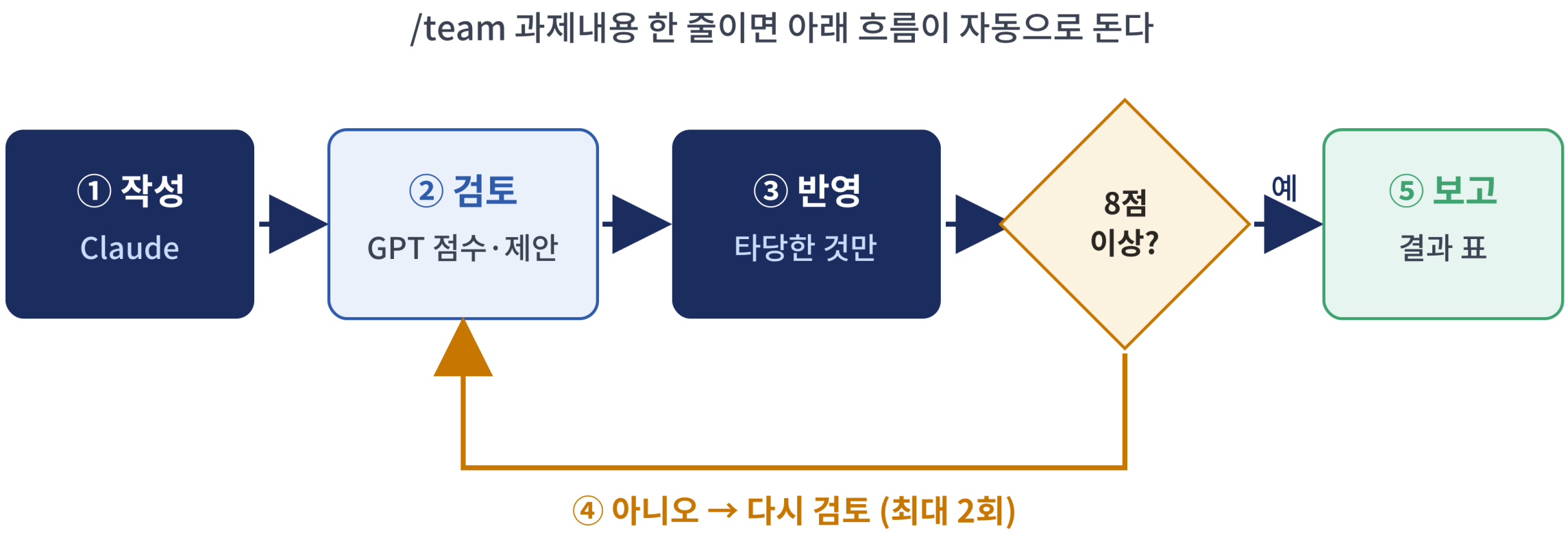
실습 3

/team 명령 만들기

작성, 검토, 반영, 보고의 분업 규칙을 슬래시 명령 하나로 저장한다

/team은 작성과 검토를 최대 두 번까지 되풀이한다

슬래시 명령 = .claude/commands 폴더의 md 파일, $ARGUMENTS 자리에 과제가 들어간다



분업 규칙을 문장으로 주면 Claude가 명령 파일을 쓴다

메타프롬프트 — 설정 파일을 직접 쓰지 않고 원하는 동작을 설명한다

[입력] 프롬프트 · Claude Code

```
.claude/commands/team.md 파일로 /team 슬래시 명령을 만들어줘. 동작 규칙은 이렇게 해줘.

1. 작성: 네(Claude)가 $ARGUMENTS 과제를 수행하고 결과물을 파일로 저장한다.
2. 검토: codex 도구로 GPT에게 과제 원문, 결과물, 평가 기준(정확성, 빠진 부분, 개선점)을
   보내고, 1~10점 점수와 수정 제안 목록을 받는다.
3. 반영: 타당한 제안만 반영해 수정한다. 반영하지 않은 제안은 이유를 한 줄로 남긴다.
4. 반복: 점수가 8점 미만이면 2~3단계를 한 번 더 한다. 최대 2회까지만 한다.
5. 보고: 최종 점수, 반영한 것, 반영하지 않은 것을 표로 보여준다.
6. 과제에 "Gemini도"라는 말이 있으면 2단계에서 Gemini CLI에도 같은 검토를 요청하고
   두 의견을 비교한다.

만든 뒤 파일 내용을 보여주고, /team이 명령 목록에 보이는지 확인해줘.
```

확인: 입력창에 /를 치면 목록에 /team이 보인다

반복 횟수에 상한을 두는 것이 설계의 핵심이다

상한이 없으면 두 AI가 사소한 표현을 두고 계속 고친다

첫 검토

개선 대부분

빠진 처리, 계산 오류, 구조 문제가 이 단계에서 나온다

두 번째 검토

마무리

첫 수정이 새 문제를 만들지 않았는지 확인한다

세 번째부터

수확 체감

표현 취향 차이만 남고 사용량과 시간만 늘어난다

8점 기준과 2회 상한은 과제 성격에 맞게 바꿔도 된다

04

실습 4

실제 과제로 팀 돌려보기

코드, 문서, 교차 검토 세 과제로 GPT가 무엇을 잡는지 본다

코드와 문서 과제는 /team 한 줄로 시작한다

과제 A는 yfinance로 데이터를 직접 내려받으므로 데이터 파일이 필요 없다

[입력] 프롬프트 · 과제 A — 코드

```
/team yfinance로 삼성전자(005930.KS) 최근 1년 종가를 받아 20일·60일 이동평균
교차 신호를 차트로 그리는 파이썬 스크립트를 만들고 실행해서 차트를 열어줘.
```

[입력] 프롬프트 · 과제 B — 문서

```
/team 우리 팀 주간회의에서 쓸 "생성형 AI 도입 시 정보보안 점검표"를
10개 항목으로 만들어 md 파일로 저장해줘.
```

코드는 실행 오류, 문서는 빠진 항목 — 검토자가 잘 잡는 지점이 다르다

검토자는 작성자가 놓친 가정을 짚는다

과제 A에서 흔히 나오는 검토 결과 예시 — 실제 문구는 매번 다르다

[출력] 결과 · /team 보고 표 (예시)

```
최종 점수: 6 → 8 (2회)
반영   : 상장 휴일로 빈 날짜 처리, 60일 미만 구간의 신호 제외, 차트 축 한글 깨짐 수정
미반영: "클래스로 재구성하라" — 실습용 단일 스크립트라 과하다고 판단
```

읽는 법

- 점수 변화로 검토 효과를 본다
- 미반영 이유가 합리적인지 본다

토론 거리

- 미반영 판단에 동의하는가
- 같은 지적을 사람이 했을까

교차 검토에서는 의견이 갈리는 지점이 가장 유익하다

과제에 "Gemini도"를 넣으면 실습 3의 6번 규칙이 작동한다

[입력] 프롬프트 · 과제 C — 교차 검토

```
/team Gemini도 함께 검토. 금리 인상이 은행주와 성장주에 미치는 영향을 비교하는
한 쪽짜리 브리핑을 써줘. 두 검토자의 의견이 다른 부분을 따로 표시해줘.
```

무엇을 보나

- 두 검토자가 같은 지적을 했다면 반영할 근거가 강하다
- 의견이 갈리면 그 쟁점이 바로 사람이 판단해야 할 지점이다
- 검토자 AI의 지적도 틀릴 수 있다 — 최종 판단은 사람이 한다

05

자동차 적용

Day 4·5 데이터로 AI 팀 돌리기

보증 판정, 견적 메일, 설비 센서, 파일럿 공정 — 검토자가 함정을 잡는지 본다

과정 데이터에는 검토자가 잡아야 할 함정이 심겨 있다

모두 실습용 가상 자료 — keerhee/kma-auto-genai-3-1 저장소의 data 폴더

예제	과정 위치	데이터	검토자가 잡아야 할 것
① 보증 판정	Day 4 업무자동화	보증판정 테스트사례 12건	렌터카 한도 단축, 최초등록일 미확인
② 견적 메일 회신	Day 4 업무자동화	딜러 견적 요청 메일 20건	권한 밖 특가 약속, 리콜 문의 혼입
③ 설비 이상탐지	Day 5 예지보전	설비센서 단위혼재 50건	°F, psi, in/s로 기록된 값
④ 파일럿 공정 선정	Day 5 예지보전	공정별 파일럿 평가 가정값	시간당 손실만 보고 매긴 순위

예제 ①~③은 GPT 한 명, 예제 ④는 GPT와 Gemini 교차 검토로 진행한다

데이터 내려받기도 Claude Code에게 맡긴다

ChatGPT 계정이 없는 수강생은 Gemini를 1차 검토자로 바꿔 같은 실습을 한다

[입력] 프롬프트 · Claude Code — 데이터 준비

```
github.com/keerhee/kma-auto-genai-3-1 저장소의 data/Day4, data/Day5 폴더를 내 작업 폴더의
data 폴더로 내려받아줘. 다 받으면 파일 목록과 각 파일의 앞 3줄을 보여줘.
```

[입력] 프롬프트 · Claude Code — ChatGPT 계정이 없을 때

```
.claude/commands/team.md에서 2단계 검토자를 codex 대신 Gemini CLI(gemini -p)로 바꿔줘.
나머지 규칙은 그대로 두고, 바꾼 부분만 보여줘.
```

Claude Pro만 있어도 된다

- Gemini CLI는 Google 계정 로그인만으로 무료 사용량 안에서 검토를 맡는다

보증 판정은 네 가지 결론 중 하나로 끝난다

한빛모터스 CS 매뉴얼 1장 — 기간과 주행거리 중 먼저 도래하는 쪽에서 보증이 끝난다

구분	기간 / 주행거리
일반 부품, 엔진·변속기	3년 / 6만 km
EV 전용 부품	8년 / 16만 km
12V 보조배터리	1년 / 2만 km
영업용 차량의 일반 부품	2년 / 4만 km

판정 결론 네 가지

- 보증 적용
- 보증 제외
- 추가 확인 — 정보가 부족할 때
- 판정 중단 — 안전·분쟁은 사람에게 연결

보증 시작일은 출고일이 아니라 최초 등록일이다

정답 열을 가린 채 만들고, 채점할 때만 정답을 쓴다

gt_로 시작하는 열이 정답이다 — 스크립트가 정답을 보면 실력이 아니라 베끼기가 된다

[입력] 프롬프트 · Claude Code

```
/team data/Day4의 한빛모터스_CS매뉴얼을 기준으로 보증 판정 파이썬 스크립트를 만들어줘.
보증판정_테스트사례12 파일의 12건에 돌려서 결론(보증 적용, 보증 제외, 추가 확인, 판정 중단)과
근거 조항을 표로 보여줘. gt_로 시작하는 정답 열은 스크립트가 읽지 말고 채점할 때만 써줘.
검토 기준: 12건 정답률, 판정 중단 2건을 하나도 놓치지 않았는지, 매뉴얼에 없는 내용을
지어내지 않았는지.
```

성공 기준

- 12건 중 12건 정답
- 판정 중단 연결 재현율 100%

검토자에게 기대하는 것

- 놓친 예외 조항을 짚는다
- 매뉴얼 밖 추측을 걸러 낸다

검토자가 짚어야 할 사례는 다섯 묶음이다

문의일 2026-10-23 기준 — 정답은 테스트사례 파일의 gt 열과 같다

사례	함정	정답
T04 렌터카, 4.5만 km	기간은 남았지만 영업용은 4만 km로 줄어든다	보증 제외
T07 출고일만 확인	보증 시작일은 최초 등록일이다	추가 확인
T10 중고 구입	차주가 바뀌어도 남은 기간이 승계된다	보증 적용
T11 앱 표시 용량 68%	용량은 서비스센터 진단으로만 판정한다	추가 확인
T08 경고등, T12 소송 언급	안전과 분쟁은 판정하지 않고 연결한다	판정 중단

판정 중단 두 건을 놓치면 정답률이 높아도 불합격이다

회신 초안은 권한 밖 약속이 없어야 쓸 수 있다

영업관리팀 공용 메일함 1주일치 20건 — 특가·재고 건은 사람이 승인해야 한다

[입력] 프롬프트 · Claude Code

```
/team data/Day4의 딜러견적요청메일20건을 읽고 메일마다 회신 초안을 써줘. 각 메일을 정상 요청,
특가 요청, 재고 없음, 사양 불명확, 법인·대량, 경쟁사 견적 첨부, 범위 밖 중 하나로 분류하고,
특가와 재고처럼 승인이 필요한 건은 초안 대신 '담당자 확인 필요'로 표시해줘.
검토 기준: 할인이나 납기를 약속한 문장이 없는지, 분류가 맞는지.
```

검토 포인트

- 11·12번 특가에 할인을 약속했나
- 20번 리콜 문의를 CS로 넘겼나

숨은 함정

- 09번은 2대지만 개인 명의 각 1대다
- 그래서 법인·대량 거래가 아니다

이상 후보를 찾기 전에 데이터부터 의심해야 한다

이상 후보 = 설비별 평균에서 표준편차의 두 배 넘게 벗어난 값

[입력] 프롬프트 · Claude Code

```
/team data/Day5의 설비센서_단위혼재_함정 파일로 설비별 이상 후보를 찾아줘. 설비A와 설비B를
따로 나눠 진동·온도·압력마다 평균에서 표준편차 두 배 넘게 벗어난 값을 표시하고,
시간 흐름 그래프를 그려서 열어줘.
검토 기준: 계산 전에 데이터 자체에 문제가 없었는지, 이상 후보가 한 점인지 흐름인지 구분했는지.
```

프롬프트에 일부러 쓰지 않은 것

- 단위 문제는 알려 주지 않는다 — 작성자와 검토자 중 누가 먼저 찾는지 본다
- 둘 다 놓치면 마지막 확인은 사람이 한다

단위가 섞인 여섯 값이 가짜 이상 신호를 만든다

단서는 '기록장치' 열뿐 — 교체 로거와 휴대용 로거가 다른 단위로 적었다

설비	시각	항목	기록값	실제 값
A	10:00	온도	123.6 °F	50.9 °C
A	10:20	온도	124.7 °F	51.5 °C
A	10:40	온도	124.0 °F	51.1 °C
B	13:20	진동	0.043 in/s	1.08 mm/s
B	13:40	진동	0.043 in/s	1.1 mm/s
B	15:00	압력	1690 psi	116.5 bar

단위를 고치고 나면 설비B 마지막 두 점의 진동 상승이 진짜 신호로 남는다 — 한 점이 아니라 흐름이다

작성자가 놓치면 검토자가, 둘 다 놓치면 사람이 기록장치 열을 본다

파일럿 공정 선정은 교차 검토로 쟁점을 드러낸다

연간 정지손실 = 시간당 손실 × 연간 고장 건수 × 평균 정지 시간 (모두 가정값)

[입력] 프롬프트 · Claude Code

```
/team Gemini도 함께 검토. data/Day5의 공정별_파일럿평가_가정값으로 예지보전 파일럿을 먼저
시작할 공정 하나를 추천하는 한 쪽짜리 보고서를 써줘. 두 검토자의 의견이 다른 부분을 따로 표시해줘.
```

공정	시간당 정지손실	연간 정지손실	초기 투자	손실 ÷ 투자
프레스	1,200만 원	6.48억 원	2.5억 원	2.59배
도장	1,500만 원	3.60억 원	3.5억 원	1.03배
차체	1,800만 원	3.46억 원	6.0억 원	0.58배
조립	2,500만 원	1.88억 원	4.0억 원	0.47배

검토자의 의견이 갈리는 곳이 사람이 판단할 곳이다

교차 검토에서 나오기 쉬운 쟁점 세 가지 — 모두 가정값에 대한 해석 차이다

시간당 손실 함정

조립이 1위처럼 보인다

시간당 2,500만 원으로 가장 크지만 고장이 연 5건이라 연간 손실은 가장 작다

재고 버퍼

프레스 손실은 과대일 수 있다

패널 재고가 있어 실제 정지 손실은 표보다 작을 수 있다

절감 비율 가정

전부 막지는 못한다

예지보전이 줄이는 손실 비율(예: 20~30%, 추정)을 따로 가정해야 순효과가 나온다

모범답안은 프레스 1순위 — 근거와 한계를 함께 적었는지가 채점 기준이다

06

심화 실습

Reason Network로 대등한 AI 팀 만들기

두 터미널의 Claude Code와 Codex가 서로 일을 주고받게 한다

Reason Network는 AI 세션들을 대등한 동료로 묶는다

reason-machines가 공개한 오픈소스(MIT/Apache) P2P 메시징 엔진 — 게시물에 소개된 도구

구분	지금까지 실습 (MCP 도구 방식)	Reason Network
구조	Claude가 지휘하고 Codex를 도구로 호출	각 AI 세션이 대등한 동료
세션	Claude 세션 하나	터미널마다 세션이 따로 실행
통신	Claude가 도구를 부른다	list_team, send_message로 서로 메시지
적합한 일	작성 후 검토 같은 단순 분업	병렬 개발, 릴레이 작업, 장시간 자율 작업

지원 에이전트: Claude Code, Codex, Gemini CLI, Cursor, Copilot CLI 등 — 상태는 내 PC의 ~/.reason/network에 저장된다

두 세션은 각자 일하다가 데몬을 거쳐 대화한다

Claude Code와 Codex는 메시지를 세션 도중에 받고, 쉬던 세션도 깨어난다



실습은 네 단계로 진행한다

사람이 직접 하는 일은 계정 선택과 두 번째 터미널을 여는 것 두 가지뿐이다

① 설치

Claude Code가 한다

공식 README대로 엔진을 설치하고 Claude Code에 MCP를 등록한다

② 계정 선택

사람이 고른다

Reason 계정으로 로그인하거나, 로그인 없이 팀 링크를 받는다

③ Codex 세션

사람이 연다

새 터미널에서 Codex를 실행해 두 번째 팀원을 띄운다

④ 메시지 실습

두 AI가 한다

Claude가 Codex에게 검토를 보내고 답을 받는다

MCP 방식과 달리 Codex 세션이 따로 떠 있지 않으면 팀원 목록에 나타나지 않는다

설치와 등록은 공식 README를 읽고 Claude Code가 한다

명령어는 버전마다 바뀔 수 있으므로 README를 먼저 읽게 시킨다 — 터미널 A에서 실행

[입력] 프롬프트 · Claude Code (터미널 A)

```
Reason Network(reason-machines)의 공식 GitHub 저장소와 README를 찾아 읽어줘.

1. README의 설치 스크립트로 엔진을 설치해줘. 내 PC가 Windows면 PowerShell 스크립트를 써줘.
2. 설치 중 Reason 계정 로그인 여부를 물으면 멈추고 나에게 물어봐. 내가 고를게.
3. Claude Code와 Codex 양쪽에 reason-network MCP 서버를 등록해줘.
   (설치 스크립트가 이미 등록했다면 확인만 해줘.)
4. 다 되면 내가 터미널 B에서 무엇을 해야 하는지 한 줄로 알려줘.
오류가 나면 원인을 찾아 직접 고치고, 무엇을 고쳤는지 한 줄로 알려줘.
```

Codex CLI는 실습 1에서 이미 설치하고 로그인해 두었다

혼자 실습할 때는 로그인 없이 팀 링크로 충분하다

설치 중 Claude Code가 멈추고 물어보면 아래 기준으로 고른다

Reason 계정으로 로그인

- 워크스페이스가 곧 팀이 된다
- 동료 PC의 세션까지 묶을 때 쓴다

로그인하지 않음

- 엔진만 설치하고 팀 링크를 만든다
- 다른 사람은 이 링크로 합류한다

수업에서는 '로그인하지 않음'을 고르고, 짝 실습을 할 때만 팀 링크를 공유한다

두 번째 터미널에서 Codex를 띄우면 팀원이 둘이 된다

터미널 B = 새 터미널 창 — 실습 폴더로 이동한 뒤 codex를 실행한다

[입력] 명령어 · 터미널 B에 입력

```
codex
```

[입력] 프롬프트 · Codex (터미널 B)

```
reason-network 도구로 list_team을
실행해서 팀원 목록을 보여줘.
```

[출력] 결과 · list_team (예시 — 실제 표기는 버전마다 다르다)

```
팀원 세션 2개
- Claude Code · keehong · kma-auto-genai-3-1 (main) · 쉼   · 대기
- Codex       · keehong · kma-auto-genai-3-1 (main) · 바쁨 · 팀원 목록 조회 중
```

확인: 두 세션이 모두 보이면 연결은 끝났다

Claude가 Codex에게 검토를 보내고 답을 받는다

자동차 예제 ①의 보증 판정 스크립트를 그대로 쓴다 — 터미널 A에 입력

[입력] 프롬프트 · Claude Code (터미널 A)

```
list_team으로 Codex 세션을 찾고, send_message로 이렇게 보내줘:
"보증 판정 스크립트를 data/Day4 보증판정 테스트사례 12건으로 검증하고, 틀린 사례 번호와
 이유, 매뉴얼에 없는 추측이 있는지를 답장해줘. gt_ 정답 열은 채점에만 써줘."
답장이 오면 그대로 보여주고, 타당한 지적만 반영해서 다시 돌린 결과를 표로 보여줘.
```

[출력] 결과 · 터미널 A가 받은 답장 (예시)

```
From: Codex (터미널 B)
12건 중 11건 정답. T04 오답 — 렌터카는 일반 부품 한도가 2년/4만 km로 줄어드는데
스크립트가 3년/6만 km를 적용함 (매뉴얼 1장). 매뉴얼 밖 추측은 없음.

→ Claude: T04 규칙 반영 후 재실행, 12건 중 12건 정답
```

터미널 B에서는 메시지가 도착하는 순간을 지켜본다

Codex 세션은 작업 도중에도 메시지를 받고, 쉬고 있었다면 깨어나서 일을 시작한다

터미널 B에서 볼 것

- 메시지가 들어오자 Codex가 스스로 작업을 시작하는가
- 작업 후 답장을 send_message로 돌려보내는가

사람이 할 것

- 두 창을 나란히 놓고 지켜본다
- 답장 내용이 타당한지 마지막에 판단한다

MCP 방식에서는 Claude가 기다렸지만, 여기서는 두 세션이 각자 일하며 주고받는다

연결 문제도 증상을 그대로 Claude Code에게 알린다

아래 프롬프트의 증상 부분만 바꿔 붙여 넣는다

[입력] 프롬프트 · Claude Code

```
Reason Network에서 [증상]이 생겼어. README를 다시 확인해서 원인을 찾고 고친 뒤 확인해줘.
```

증상	흔한 원인
list_team에 Codex가 없다	터미널 B에서 Codex를 실행하지 않았거나 MCP 미등록
메시지가 도착하지 않는다	푸시를 못 받는 에이전트 — check_inbox로 직접 확인
설치 스크립트가 멈췄다	계정 선택 질문에 답하지 않았다
다른 사람이 합류하지 못한다	팀 링크를 공유하지 않았거나 방화벽이 막았다

주의

검토를 보내는 순간 내용은 외부 서버로 나간다

보안, 비용, 판단 책임 세 가지를 지킨다

보안

가장 중요

회사 기밀, 고객 정보, 개인정보는 검토 요청에 넣지 않는다. OpenAI와 Google 서버로도 전송된다

비용

사용량 한도

구독 로그인은 사용량 한도가 있고, API 키는 호출마다 과금된다. 반복 상한을 둔 이유다

판단

사람의 몫

검토자의 점수는 참고 지표일 뿐이다. 최종 결정과 책임은 사람이 진다

보안 규칙은 CLAUDE.md에 적어 두면 Claude가 스스로 지킨다

실습 결과를 두고 네 가지를 이야기한다

Q1	GPT가 지적했지만 Claude가 반영하지 않은 제안이 있었나? 그 판단은 옳았나?	**판단 검증**
Q2	과제 C에서 두 검토자의 의견이 갈린 쟁점은 무엇이었나?	**교차 검토**
Q3	우리 업무에서 검토자를 붙이면 효과가 클 작업은 무엇인가?	**업무 적용**
Q4	외부 AI에 보내면 안 되는 자료를 우리 팀은 어떻게 구분하는가?	**보안**

AI 팀은 연결, 규칙, 판단의 세 층으로 이루어진다

1

연결 — MCP와 Bash 호출

Codex는 MCP 서버로, Gemini는 gemini -p로 Claude Code에 붙였다.

2

규칙 — /team과 CLAUDE.md

분업 순서, 점수 기준, 반복 상한, 보안 규칙을 파일로 저장했다.

3

판단 — 사람의 몫

검토 의견을 받아들일지와 외부로 보낼 자료의 범위는 사람이 정한다.

다음 단계: 자기 업무 과제 하나를 골라 /team으로 돌려 본다